Final Exam: STAT 5860

Gular Abbasli | Professor: Dr. Kevin H. Lee

04/30/2025

I certify here that the work on this exam is solely mine. I did not receive any assistance from others and I did not provide any assistance to others.

PRINT YOUR NAME: Gular Abbasli DATE: 04/30/2025

Problem 1: Gibbs Sampling to Estimate the Mean of a Beta-Binomial Distribution

Part 1 - Generating 10,000 Gibbs Samples from the Bivariate Density ($n = 16$, $\alpha = 2$, $\beta = 4$)

The two conditional distributions are:

- $f(x \mid y) \sim \text{Binomial}(n = 16,\ p = y)$
- $f(y \mid x) \sim \text{Beta}(x + \alpha,\ n - x + \beta) = \text{Beta}(x + 2,\ 16 - x + 4)$

We initialize the chain at $(x, y) = (5, 0)$ and iterate 10,000 times.

```
set.seed(430)

n_samples <- 10000
n         <- 16
alpha     <- 2
beta      <- 4

X_gibbs      <- matrix(0, nrow = n_samples, ncol = 2)
X_gibbs[1, ] <- c(5, 0)

for (i in 1:(n_samples - 1)) {
  x_new             <- rbinom(1, size = n, prob = X_gibbs[i, 2])
  X_gibbs[i + 1, 1] <- x_new
  y_new             <- rbeta(1, shape1 = x_new + alpha,
                                shape2 = n - x_new + beta)
  X_gibbs[i + 1, 2] <- y_new
}
```

Part 2 - Estimating the Mean of the Marginal Distribution of X from the Gibbs Sample

```
sample_mean_x <- mean(X_gibbs[, 1])
sample_mean_x
```

```
## [1] 5.3023
```

Part 3 - Comparing the Sample Mean with the Theoretical Beta-Binomial Mean

The theoretical mean of the Beta-Binomial distribution is:

$$\frac{n\alpha}{\alpha+\beta} = \frac{16 \times 2}{2+4} = \frac{32}{6} \approx 5.3333$$

```
theoretical_mean <- n * alpha / (alpha + beta)
theoretical_mean
```

```
## [1] 5.333333
```

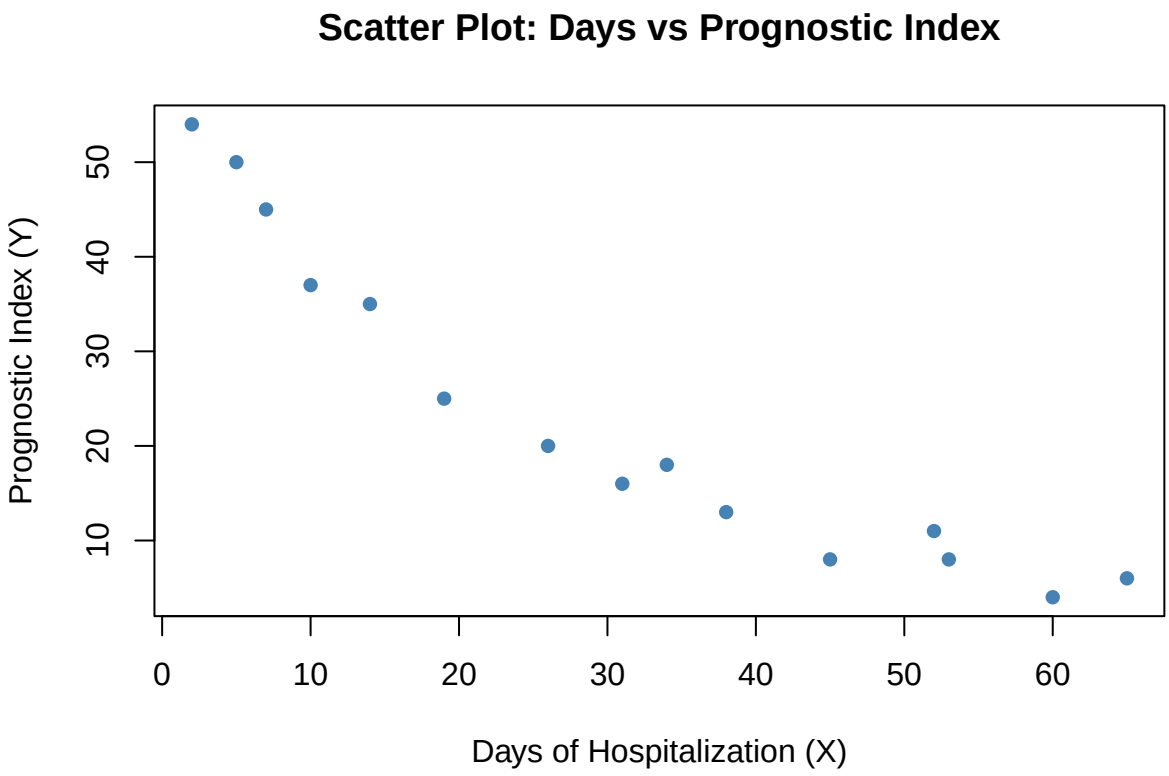
The sample mean (5.3023) is very close to the theoretical mean (5.3333), confirming the Gibbs sampler correctly targets the marginal distribution of X.

Problem 2: Nonlinear Exponential Regression for Predicting Long-Term Recovery of Hospital Patients

Part 1 - Loading the Hospital Data and Plotting Days of Hospitalization vs. Prognostic Index

```
set.seed(430)
hospital <- read.table("hospital.txt", header = TRUE)

plot(hospital$days, hospital$prog,
     xlab = "Days of Hospitalization (X)",
     ylab = "Prognostic Index (Y)",
     main = "Scatter Plot: Days vs Prognostic Index",
     pch  = 16, col = "steelblue")
```



The scatter plot shows a clear decreasing nonlinear relationship, suggesting an exponential decay model $Y_i = \beta_0 e^{\beta_1 X_i} + \varepsilon_i$ is appropriate.

Part 2 - Estimating β_0 and β_1 with Newton's Method (Own Code, Without `optim()`)

We minimize $Q = \sum_{i=1}^{n} (Y_i - \beta_0 e^{\beta_1 X_i})^2$. Let $r_i = Y_i - \beta_0 e^{\beta_1 X_i}$. The gradient and Hessian are:

$$\frac{\partial Q}{\partial \beta_0} = -2 \sum r_i e^{\beta_1 X_i}, \qquad \frac{\partial Q}{\partial \beta_1} = -2 \sum r_i \beta_0 X_i e^{\beta_1 X_i}$$

$$\frac{\partial^2 Q}{\partial \beta_0^2} = 2 \sum e^{2\beta_1 X_i}, \qquad \frac{\partial^2 Q}{\partial \beta_1^2} = 2 \sum (\beta_0 X_i e^{\beta_1 X_i})^2 - 2 \sum r_i \beta_0 X_i^2 e^{\beta_1 X_i}$$

$$\frac{\partial^2 Q}{\partial \beta_0 \partial \beta_1} = 2 \sum \beta_0 X_i e^{2\beta_1 X_i} - 2 \sum r_i X_i e^{\beta_1 X_i}$$

Newton update: $\beta_{\text{new}} = \beta - \mathbf{H}^{-1}\mathbf{g}$

```
days <- hospital$days
prog <- hospital$prog

grad_Q <- function(beta0, beta1, x, y) {
  r    <- y - beta0 * exp(beta1 * x)
```

```
  g    <- numeric(2)
  g[1] <- -2 * sum(r * exp(beta1 * x))
  g[2] <- -2 * sum(r * beta0 * x * exp(beta1 * x))
  return(g)
}

hess_Q <- function(beta0, beta1, x, y) {
  r      <- y - beta0 * exp(beta1 * x)
  H      <- matrix(0, 2, 2)
  H[1,1] <- 2 * sum(exp(2 * beta1 * x))
  H[1,2] <- 2 * sum(beta0 * x * exp(2 * beta1 * x)) -
            2 * sum(r * x * exp(beta1 * x))
  H[2,1] <- H[1,2]
  H[2,2] <- 2 * sum((beta0 * x * exp(beta1 * x))^2) -
            2 * sum(r * beta0 * x^2 * exp(beta1 * x))
  return(H)
}

beta <- c(50, 0)

for (i in 1:20) {
  g        <- grad_Q(beta[1], beta[2], days, prog)
  H        <- hess_Q(beta[1], beta[2], days, prog)
  beta_new <- beta - solve(H) %*% g
  print(beta_new, digits = 7)
  if (max(abs(beta_new - beta)) < 1e-8) break
  beta <- beta_new
}
```

```
##             [,1]
## [1,] 62.60559628
## [2,] -0.01473299
##             [,1]
## [1,] 31.87728906
## [2,] -0.01120967
##            [,1]
## [1,] 54.9175468
## [2,] -0.0332881
##             [,1]
## [1,] 58.15190424
## [2,] -0.03845394
##             [,1]
## [1,] 58.58836905
## [2,] -0.03953515
##             [,1]
## [1,] 58.60652677
## [2,] -0.03958634
##             [,1]
## [1,] 58.60656630
## [2,] -0.03958645
##             [,1]
## [1,] 58.60656631
## [2,] -0.03958645
```

Newton's Method converged after 8 iterations: $\hat{\beta}_0 \approx 58.6066$, $\hat{\beta}_1 \approx -0.039586$.

Part 3 - Estimating β_0 and β_1 with the `optim()` Function

```
Q_func <- function(params) {
  beta0 <- params[1]; beta1 <- params[2]
  sum((prog - beta0 * exp(beta1 * days))^2)
}

opt_result <- optim(c(50, 0), Q_func)
opt_result$par
```

```
## [1] 58.62806555 -0.03963194
```

`optim()` gives $\hat{\beta}_0 \approx 58.628$ and $\hat{\beta}_1 \approx -0.03963$, consistent with Newton's Method.

Part 4 - Comparing Newton's Method and `optim()` Estimates with `nls()` Output

```
nls_fit <- nls(prog ~ beta0 * exp(beta1 * days),
               data  = hospital,
               start = list(beta0 = 50, beta1 = 0))
summary(nls_fit)
```

```
##
## Formula: prog ~ beta0 * exp(beta1 * days)
##
## Parameters:
##        Estimate Std. Error t value Pr(>|t|)
## beta0 58.606535   1.472159   39.81 5.70e-15 ***
## beta1 -0.039586   0.001711  -23.13 6.01e-12 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.951 on 13 degrees of freedom
##
## Number of iterations to convergence: 6
## Achieved convergence tolerance: 8.115e-06
```

```
coef(nls_fit)
```

```
##      beta0      beta1
## 58.6065348 -0.0395864
```

`nls()` gives $\hat{\beta}_0 = 58.6065$ and $\hat{\beta}_1 = -0.039586$, matching Newton's Method to 6 significant figures. All three approaches agree.

Problem 3: Solving a Linear Programming Problem

Maximizing $4x + 2y + 9z$ Subject to Linear Constraints Using `solveLP()` from the `linprog` Package

$$\text{Maximize } 4x + 2y + 9z$$
$$\text{subject to } \quad 2x + y + z \leq 2, \quad -x + y - 3z \geq -3, \quad x, y, z \geq 0$$

```
set.seed(430)
library(linprog)
```

```
## Warning: package 'linprog' was built under R version 4.5.2
```

```
## Loading required package: lpSolve
```

```
cvec      <- c(4, 2, 9)
Amat      <- matrix(c( 2,  1,  1,
                      -1,  1, -3), nrow = 2, byrow = TRUE)
bvec      <- c(2, -3)
const.dir <- c("<=", ">=")

LP_result <- solveLP(cvec, bvec, Amat,
                     maximum   = TRUE,
                     const.dir = const.dir,
                     lpSolve   = TRUE)
LP_result$solution   # x = 0, y = 0.75, z = 1.25
```

```
##    1    2    3
## 0.00 0.75 1.25
```

```
LP_result$opt        # Maximum = 12.75
```

```
## [1] 12.75
```

The optimal solution is $x = 0,\ y = 0.75,\ z = 1.25$ with maximum value $4(0) + 2(0.75) + 9(1.25) = \mathbf{12.75}$.

Constraint	Value	Satisfied?
$2x + y + z \leq 2$	$0 + 0.75 + 1.25 = 2$	✓
$-x + y - 3z \geq -3$	$0 + 0.75 - 3.75 = -3$	✓
$x,\ y,\ z \geq 0$	$0,\ 0.75,\ 1.25$	✓

Problem 4: Bootstrap Inference for the Rate Parameter of an Exponential Distribution (Job Submission Times)

Part 1 - Bootstrap Standard Error of the MLE $\hat{\lambda}$ (5,000 Replicates, Own Code, Without `boot()`)

The MLE is $\hat{\lambda} = n \big/ \sum_{i=1}^{n} x_i$.

```r
set.seed(430)

job <- read.table("jobsubmission.txt", header = TRUE)
x   <- job$x
n   <- length(x)   # n = 20

lambda_hat <- n / sum(x)
lambda_hat
```

```
## [1] 4.049161
```

```r
B           <- 5000
lambda_boot <- numeric(B)

for (b in 1:B) {
  x_boot         <- sample(x, size = n, replace = TRUE)
  lambda_boot[b] <- n / sum(x_boot)
}

se_boot <- sd(lambda_boot)
se_boot
```

```
## [1] 0.8729568
```

The MLE is $\hat{\lambda} \approx 4.049$ and the bootstrap standard error is approximately **0.873**.

Part 2 - 95% Percentile Bootstrap Confidence Interval for the MLE $\hat{\lambda}$

```r
ci <- quantile(lambda_boot, c(0.025, 0.975))
ci
```

```
##     2.5%    97.5%
## 2.885600 6.267592
```

The 95% percentile bootstrap confidence interval for λ is approximately [2.886, 6.268]. This requires no distributional assumptions and is based entirely on the empirical quantiles of the bootstrap distribution of $\hat{\lambda}$.